
FINAL PROJECT REPORT

Implement Client Script & UI Policy (Incident)

A ServiceNow Micro Project on Dynamic Form Behaviour, Data Validation and Automation

Project Title	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow
Module Scope	UI Policies, UI Policy Actions, Client Scripts (onChange, onSubmit, onCellEdit)
Table Used	Incident
Report Format	Final Report Template

1. INTRODUCTION

1.1 Project Overview

This micro project, **Implement Client Script & UI Policy (Incident)**, is built on the ServiceNow platform and focuses on the **Incident** table. It demonstrates how **UI Policies** and **Client Scripts** work together to enforce dynamic field behaviour, automate field updates, and prevent incorrect data submission on Incident forms. All configurations respond dynamically to the value of the **Impact** field, ensuring fields like **Assigned To** and **Urgency** behave correctly when an Incident is High impact.

1.2 Purpose

The purpose of this project is to showcase practical best practices for improving form usability and data integrity in ServiceNow. By combining `onChange`, `onSubmit`, and `onCellEdit` Client Scripts with UI Policy actions, the project enforces mandatory fields, controls user interactions, automates updates, and keeps Incident data clean and consistent — while remaining lightweight, efficient, and easy to implement for a short-duration micro project.

2. IDEATION PHASE

2.1 Problem Statement

In a ServiceNow Incident management process, agents can accidentally save critical (High impact) Incidents without assigning them, change the Urgency inconsistently with Impact, or update the State directly from a list view — bypassing form validations. This leads to poor data quality, lack of accountability for critical incidents, and inconsistent ticket lifecycle handling. There is a need for an automated, form-level control mechanism that enforces correct field behaviour based on the Impact of the Incident.

2.2 Empathy Map Canvas

The empathy map below captures the perspective of an IT support agent (the primary user of the Incident form) while handling High impact incidents:

SAYS "I need to log critical incidents quickly." "Sometimes I forget to assign the ticket immediately."	THINKS "High impact tickets must be handled carefully." "The system should guide me when data is missing."
DOES Logs the incident, sets Impact = High, tries to save quickly. Occasionally edits State directly from the list for speed.	FEELS Pressured during major incidents. Frustrated when invalid data is saved and needs rework later.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	Agent Action	System Behaviour
-------	--------------	------------------

Log Incident	Creates a new Incident and sets Impact = High	UI Policy triggers; Assignment group becomes mandatory; Urgency auto-sets to High
Fill Details	Enters incident details; Urgency shown read-only	UI Policy Action keeps Urgency read-only for High impact
Attempt Save	Clicks Submit (may leave Assigned To empty)	onSubmit script blocks save with an error message
Correct & Save	Fills Assigned To and submits	Record saves successfully with consistent data
Update State	Tries list edit of State; then updates via form	onCellEdit blocks list edit; form update allowed

3.2 Solution Requirement

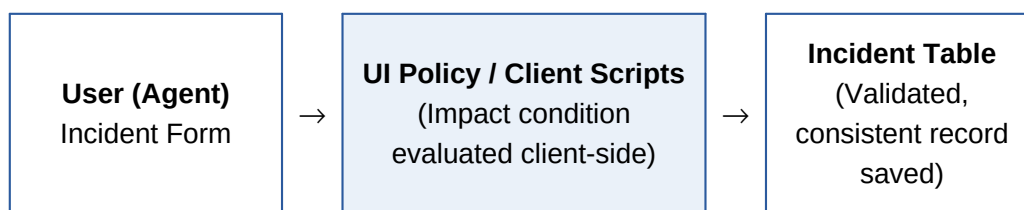
Functional Requirements:

- FR1: Assignment group must become mandatory when Impact = High (UI Policy).
- FR2: Urgency must become read-only when Impact = High (UI Policy Action).
- FR3: Urgency must auto-set to High when Impact changes to High (onChange Client Script).
- FR4: Saving must be blocked when Impact = High and Assigned To is empty (onSubmit Client Script).
- FR5: State must not be editable via list edit (onCellEdit Client Script).
- FR6: All enforced behaviours must reverse automatically when Impact is no longer High (Reverse if false).

Non-Functional Requirements:

- NFR1: Lightweight configuration — no server-side code, instant client-side execution.
- NFR2: Easy to implement and maintain within a short-duration micro project.
- NFR3: Clear user feedback through info and error messages.

3.3 Data Flow Diagram



3.4 Technology Stack

Platform	ServiceNow (SaaS ITSM platform)
Application	Incident Management module
Configuration Layer	System UI → UI Policies, UI Policy Actions
Scripting	Client-side JavaScript — Client Scripts (onChange, onSubmit, onCellEdit) using g_form APIs

Key APIs Used	g_form.setValue(), g_form.getValue(), g_form.addInfoMessage(), g_form.showErrorMessage()
No-Code Elements	UI Policy conditions and actions (Mandatory, Read-only, Visible), Reverse if false

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	Solution Fit
High impact incidents saved without an assignee	onSubmit Client Script blocks the save and shows an error on Assigned To
Urgency changed inconsistently with Impact	onChange script auto-sets Urgency; UI Policy Action makes it read-only
Assignment group skipped for critical tickets	UI Policy makes Assignment group mandatory when Impact = High
State edited directly in list view, bypassing rules	onCellEdit Client Script cancels list edits of State with an alert

4.2 Proposed Solution

The solution uses five coordinated configurations on the Incident table, all keyed to the Impact field:

UI Policy	"High Impact Control" — Impact is 1-High → Assignment group Mandatory; Reverse if false = true
UI Policy Action	Urgency → Read-only = true when policy is active
Client Script 1	onChange on Impact → sets Urgency = 1-High with info message
Client Script 2	onSubmit → returns false if Impact = 1 and Assigned To is empty
Client Script 3	onCellEdit on State → alerts and calls callback(false) to block list edits

4.3 Solution Architecture

Presentation Layer Incident Form & List View (Browser)	Control Layer UI Policy + UI Policy Actions Client Scripts (onChange / onSubmit / onCellEdit)
Data Layer Incident Table (validated, consistent records)	Platform Layer ServiceNow Instance (System UI module)

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Activity	Duration
1. Planning	Understand requirements, design policy/script logic	1 day
2. Configuration	Create UI Policy, UI Policy Action and 3 Client Scripts	2 days
3. Testing	Execute 5 test cases; verify enforcement and reverse behaviour	1 day
4. Documentation	Prepare final report with screenshots and source code	1 day

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

All validations execute client-side in the browser, so no server round-trips are required for field enforcement. The scripts use minimal API calls (single g_form operations) and include guard clauses (e.g., isLoading check in onChange), ensuring instant response with negligible performance impact on the form.

Test Cases Executed:

TC	Scenario	Expected Result	Status
TC1	Save High impact Incident with empty Assigned To	Save blocked; error box shown on Assigned To	PASS
TC2	Save High impact Incident with Assigned To filled	Record saved; Urgency auto-set and read-only	PASS
TC3	Change Impact from High to Medium	Mandatory and read-only behaviours reversed; save succeeds	PASS
TC4	Double-click State in list view to edit	Alert shown; State unchanged	PASS
TC5	Change State via Incident form	State updated and saved successfully	PASS

7. RESULTS

7.1 Output Screenshots

The configured behaviours were verified on the ServiceNow instance with the following outcomes:

Result	Observation
UI Policy enforcement	Assignment group became mandatory and Urgency became read-only when Impact was set to High
onChange automation	Urgency automatically set to High with info message: "Urgency set to High for High impact incident."

onSubmit validation	Save blocked with error box: "Assigned To is mandatory for High impact incidents."
onCellEdit restriction	Alert shown on list edit: "State cannot be updated using list editing. Please open the Incident."
Reverse behaviour	All enforced behaviours reversed correctly when Impact changed from High to Medium; record saved successfully
Form State update	State changed and saved successfully via the Incident form

Outcome: All configured behaviours worked as expected — mandatory field enforcement, read-only control, automated urgency updates, save validation, and list-edit restriction — with correct reversal when Impact was changed away from High.

8. ADVANTAGES & DISADVANTAGES

Advantages:

- Improved data integrity — invalid High impact Incidents cannot be saved.
- Automation reduces manual effort and human error (Urgency auto-set).
- Enforces accountability — Assigned To and Assignment group required for critical tickets.
- Client-side execution — instant feedback, no server load.
- Lightweight and easy to implement; no custom tables or server scripts needed.
- Reversible behaviour keeps the form flexible for non-High impact incidents.

Disadvantages:

- Client-side controls can technically be bypassed by advanced users (mitigated by ACLs / Data Policies in full deployments).
- Logic is tied to field values (Impact = '1'); choice value changes require script updates.
- onCellEdit blocks all State list edits without exception rules.

9. CONCLUSION

The **Implement Client Script & UI Policy (Incident)** micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behaviour, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like **Assigned To** and **Urgency** respond appropriately to changes in **Impact**, maintaining clean and consistent data. The solution is lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.

10. FUTURE SCOPE

- Extend the policy pattern to other tables (Problem, Change Request) and other impact levels.
- Add Data Policies / ACLs as a server-side safety net alongside client-side scripts.

- Introduce exception rules for the State restriction (e.g., allow specific roles to list-edit).
- Send notifications or trigger Flow Designer workflows when a High impact Incident is assigned.
- Build a dashboard/report to monitor High impact Incidents missing assignments.

11. APPENDIX

Source Code (if any)

A. onChange Client Script — Auto set urgency for high impact:

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

B. onSubmit Client Script — Prevent save if Assigned To missing:

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

C. onCellEdit Client Script — Prevent state change via list edit:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');
```

Dataset Link

Not applicable — this project uses standard Incident table data on a ServiceNow instance. No external dataset is required.

GitHub & Project Demo Link

GitHub Repository: [Add your repository link here]

Project Demo Video: [Add your demo video link here]

ServiceNow Instance (demo): [Add your instance/developer site link here]

End of Final Report — Implement Client Script & UI Policy (Incident)